DDoS Attack Detection Using Deep Learning

Project report | LSTM, TensorFlow, FastAPI | CIC-DDoS2019 | November 2026

Submitted by: C.Rahul Reddy | **Roll No:** 2024091949

Abstract

Distributed Denial of Service (DDoS) attacks make online services unavailable by flooding them with traffic from many sources, and fixed rules struggle to recognise new attack patterns. This project builds an intelligent detector that classifies network flows as benign or DDoS attack using a Long Short-Term Memory (LSTM) deep learning model trained on the CIC-DDoS2019 dataset. The work covers data preprocessing, feature scaling, class balancing and model evaluation, and the trained model is deployed through a FastAPI service for real-time traffic classification. This report presents the proposed architecture and workflow, the algorithms used to detect attacks, the outputs to capture as evidence, and the conclusions.

1. Introduction

1.1 Background

A DDoS attack floods a target with requests from many compromised machines until legitimate users can no longer be served. Volumetric attacks such as UDP floods and reflection/amplification attacks through DNS, NTP, LDAP, SSDP and similar services can saturate bandwidth within seconds. Rule-based defences, such as rate limits and static signatures, need constant manual updates and miss attack variants they were not written for.

1.2 Problem statement

Given the statistical description of a network flow (packet counts, byte counts, timing and flag features), decide whether the flow is benign or part of a DDoS attack, fast enough to be used in real time.

1.3 Objectives

- Build an LSTM-based classifier for BENIGN and ATTACK flows.

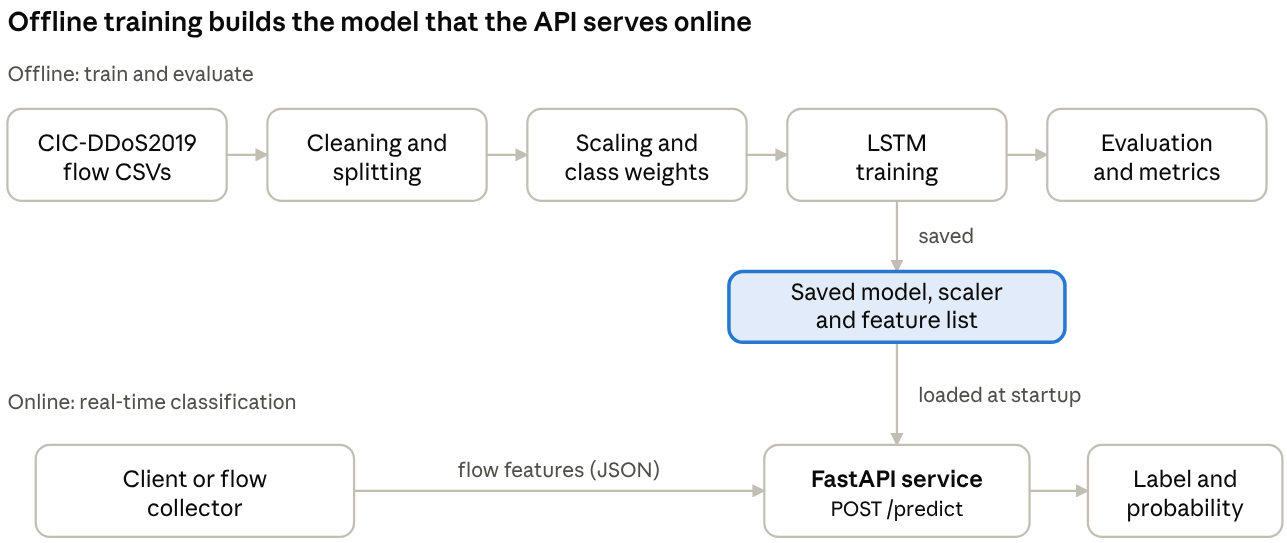
- Prepare the data correctly: cleaning, leakage prevention, feature scaling and class balancing.
- Evaluate the model with metrics suited to imbalanced data and compare it with classical baselines.
- Deploy the model as a REST API for real-time flow classification.

1.4 Dataset

The project uses CIC-DDoS2019, created by the Canadian Institute for Cybersecurity at the University of New Brunswick. It contains benign traffic and modern DDoS attacks (for example DrDoS_DNS, DrDoS_NTP, Syn, UDP and UDPLag) captured in a testbed and exported as labelled flow records with about 80 numeric features per flow. Attack flows greatly outnumber benign flows, so class imbalance has to be handled. The files are tens of gigabytes, so a configurable fraction of each file is sampled.

2. Proposed Architecture

2.1 Architecture diagram



architecture · offline training pipeline and online serving

The offline pipeline turns labelled flow records into a trained model and scaler; the online service loads both and classifies each incoming flow.

2.2 Explanation of the components

Component	Role
Flow CSVs (CIC-DDoS2019)	Labelled flow records, one row per connection, used for training and testing
Cleaning and splitting	Removes identifiers, inf, NaN and duplicates; creates the binary label; makes a stratified 70/15/15 split
Scaling and class weights	StandardScaler fitted on training data only; clipping to ±10; class weights for the minority class
LSTM training	Two stacked LSTM layers with dropout and a sigmoid output; early stopping on validation loss
Evaluation	Precision, recall, F1, ROC-AUC, benign recall, confusion matrix and latency
Saved model, scaler and feature list	Files the API loads so serving matches training exactly
FastAPI service	Validates input, applies the same scaling, runs the model, returns label and probability
Client or flow collector	Sends flow features as JSON; in a live setup a tool such as CICFlowMeter produces them

2.3 Work flow

1. Download the CIC-DDoS2019 flow CSVs and sample a fraction of each file.
2. Strip column names, drop identifier columns (IPs, ports, timestamp, Flow ID), and remove inf, NaN and duplicate rows.
3. Label flows: BENIGN = 0, any attack family = 1.
4. Split into training, validation and test sets with the same class ratio in each.
5. Fit the scaler on the training set, scale all sets and clip extreme values.
6. Compute class weights from the training labels.
7. Train the LSTM with early stopping and learning-rate reduction.
8. Evaluate on the held-out test set and compare with the baseline models.
9. Save the model, scaler and feature list.

10. Start the FastAPI service; clients send flow features to /predict or /predict_batch.
11. For each flow the service returns BENIGN or ATTACK with an attack probability.

3. Techniques and Algorithms Used to Detect Attacks

3.1 LSTM classifier

A Long Short-Term Memory network is a recurrent neural network whose cells keep a memory (the cell state) controlled by three gates. At each step the gates decide what to forget, what to store and what to output:

```
f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)
i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)
\tilde{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)
c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t
o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)
h_t = o_t \odot \tanh(c_t)
```

Here `x_t` is the input, `h_t` the hidden state, `c_t` the cell state, `f_t`, `i_t` and `o_t` the forget, input and output gates, and σ the sigmoid function. The last hidden state feeds dense layers that output the attack probability.

3.2 Model configuration

Layer	Configuration
Input	One time step holding the flow's scaled features (1 × F), or the F features as a sequence (F × 1)
LSTM 1	64 units, returns the full sequence
Dropout	0.3
LSTM 2	32 units
Dropout	0.3
Dense	32 units, ReLU
Output	1 unit, sigmoid: probability that the flow is an attack

Training uses the Adam optimiser (learning rate 0.001), binary cross-entropy loss, batch size 512 and up to 30 epochs.

3.3 Supporting techniques

Technique	Purpose
Identifier removal	Stops the model memorising IP addresses and ports instead of traffic behaviour
StandardScaler and clipping	Puts features on one scale and limits extreme outliers; fitted on training data only
Class weights	Makes errors on the rare BENIGN class cost more, so the model does not label everything as attack
Dropout	Reduces overfitting
Early stopping and learning-rate reduction	Stops training when validation loss stops improving and keeps the best weights
Binary cross-entropy with Adam	Standard loss and optimiser for a two-class probability output

3.4 Detection rule

For every flow the model outputs an attack probability p. If p is at least 0.5 the flow is reported as ATTACK, otherwise as BENIGN. The threshold is stored with the model and can be lowered to catch more attacks at the cost of more false alarms.

3.5 Baselines for comparison

Logistic Regression, Random Forest and an MLP are trained on the same split. Comparing them with the LSTM on precision, recall, F1, ROC-AUC and benign recall shows whether the recurrent model adds value over simpler methods.

4. Screenshots of Outputs

Capture the outputs below after running the pipeline on the full dataset and insert each screenshot under its figure number. Each row names the command or file that produces it.

Figure	Output to capture	How to produce it
Figure 1	Preprocessing console: label counts, features kept, split sizes	python -m src.preprocess --frac 0.05
Figure 2	LSTM training log with loss, precision, recall and AUC per epoch	python -m src.train
Figure 3	Training and validation loss curve	reports/loss_curve.png
Figure 4	Confusion matrix on the test set	reports/confusion_matrix.png
Figure 5	ROC curve	reports/roc_curve.png
Figure 6	Classification report and metrics	python -m src.evaluate (reports/metrics.json)
Figure 7	Baseline comparison scores	python -m src.baselines (reports/baselines.json)
Figure 8	FastAPI interactive documentation page	open http://127.0.0.1:8000/docs
Figure 9	Prediction responses from the running API	python -m tests.client

The /predict response has this format; the values come from your own run:

```
{"label": "ATTACK", "attack_probability": <0 to 1>, "latency_ms":
<milliseconds>}
```

5. Conclusion

The project delivers a complete pipeline from raw CIC-DDoS2019 flows to a deployed detection API. An LSTM classifies flows as benign or attack after preprocessing that avoids common pitfalls: leakage from identifier columns, scaling fitted on all data, duplicate rows, and accuracy reported on imbalanced classes. The FastAPI service reproduces the training-time preprocessing exactly, so predictions at serving time match those during training. Once the pipeline is run on the full dataset and the output figures are added, the project provides an honestly evaluated DDoS detector. Its main limits are the lab-generated dataset, flow-level rather than packet-level input, and the need for a flow generator in live use. Future work includes windows of consecutive flows, multi-class identification of attack families, threshold tuning and cross-day testing.

References

- Sharafaldin, I., Lashkari, A. H., Hakak, S., Ghorbani, A. A. (2019). Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy. IEEE International Carnahan Conference on Security Technology.
- Hochreiter, S., Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8).
- Chawla, N. V., Bowyer, K. W., Hall, L. O., Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16.